Computer Systems Security

Beatriz Morais Laranjinha 79262[a], Guilherme Glória Vaz Neto 76803[a]

Professor

Henrique Domingos

FACULDADE DE CIÊNCIAS E TECNOLOGIA

November 1, 2026

[a]*NOVA School of Science and Technology,*

---

1. Introduction and Architecture

The goal of the 3S project is to secure the communication between the **Secure Proxy** and the **Secure Stream Server**.

The browser requests a video from the Proxy, which communicates with the Stream Server to receive encrypted MPEG-4 media blocks. The Proxy authenticates, decrypts and forwards the recovered video to the browser.

The project is implemented in Go and divided into five main parts:

- `stream-proxy`: handles browser requests, secure communication and media decryption;
- `stream-server`: processes requests and encrypts media blocks;
- `crypto`: implements X25519, HKDF-SHA256 and AES-GCM;
- `config`: reads and validates XML cryptographic settings;
- `protocol`: implements handshake messages, Ed25519 authentication and error alerts.

Communication starts with an authenticated key exchange, followed by a three-message handshake. Media blocks are transmitted using Type 4 messages, while Type 5 signals the end of the stream. The Authentication Server belongs to Work Assignment 2.

2. Cryptographic Design

2.1. Key Exchange

The Secure Proxy and Secure Stream Server generate ephemeral X25519 key pairs and exchange their public keys.

The exchange is authenticated using Ed25519 digital signatures. Each endpoint signs the exchanged key information using its private identity key, and the other verifies the signature with a previously trusted public key. This prevents undetected public-key substitution during man-in-the-middle attacks.

Both endpoints calculate the same X25519 shared secret, which is processed using HKDF-SHA256 to derive session keys:

$$\text{X25519 Shared Secret} \rightarrow \text{HKDF-SHA256} \rightarrow \text{Session Keys}$$

A 256-bit key protects the handshake and error alerts. Media keys are derived according to the configured AES key size: 128, 192 or 256 bits.

New ephemeral X25519 keys are generated for each connection, providing independent session keys. Authentication must succeed before the streaming handshake proceeds.

2.2. AES-GCM

AES-128-GCM, AES-192-GCM and AES-256-GCM are supported for media encryption. AES-256-GCM protects the handshake and authenticated error alerts.

AES-GCM is an authenticated encryption mode (AEAD), providing confidentiality, integrity and authenticity without a separate MAC.

Each encryption uses a 96-bit nonce and generates a 128-bit authentication tag. Authentication fails if protected data is modified or an incorrect key or nonce is used.

Media sequence numbers are authenticated as associated data (AAD). The Proxy also checks the expected sequence number to reject replayed, duplicated or out-of-order blocks.

2.3. Cryptographic Configuration

The Proxy and Stream Server use XML configuration files to define cryptographic parameters.

The validated configurations are:

Parameter	**Value**
Algorithm	AES
Mode	GCM
Key size	128, 192 or 256 bits
Nonce size	96 bits
Padding	NULL
Separate MAC	NULL

The Proxy proposes its configuration during the handshake, and the Stream Server checks compatibility with its own configuration. Incompatible configurations produce an authenticated Type 255 alert.

All three AES-GCM configurations were successfully tested without recompilation. In every case, the complete 25,941,346-byte video was recovered without modification.

3. Secure Streaming Protocol

All protocol messages follow the format:

$$\boxed{\text{Header} \parallel \text{Payload}}$$

The plaintext header identifies the protocol version and release (1:1), message type and payload size. The payload contains the data specific to each message.

3.1. Protocol Flow

Figure 1 illustrates the authenticated key exchange, handshake and encrypted media transmission.

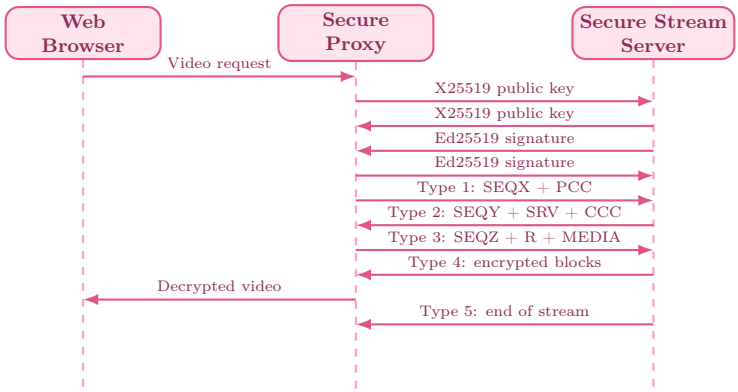


Figure 1: Authenticated Secure Streaming Protocol flow. Type 255 alerts signal protocol errors.

3.2. Type 1 – Proposal

The Proxy starts the handshake with:

$$\texttt{SEQX} \parallel \texttt{PCC}$$

`SEQX` is the initial sequence number, and PCC is the proposed cryptographic configuration.

The Stream Server receives and parses this message using the protocol functions, including `ParseHandshakeType1()`.

3.3. Type 2 – Confirmation and Challenge

After validating the proposal, the Stream Server sends:

$$\texttt{SEQY} \parallel \texttt{SRV} \parallel \texttt{CCC}$$

Here, $SEQY = SEQX + 1$, SRV is a random 256-bit challenge and CCC confirms the cryptographic configuration.

The Type 2 payload is encrypted and authenticated using AES-256-GCM with the derived session key.

3.4. Type 3 – Challenge Response and Media Request

The Proxy responds with:

$$\texttt{SEQZ} \parallel \texttt{R} \parallel \texttt{MEDIA}$$

where $SEQZ = SEQY + 1$ and $R = SHA256(SRV)$.

The Stream Server validates the sequence number and challenge response before processing the requested media filename. Filename validation also protects against path traversal and symlink escapes.

3.5. Type 4 – Media Blocks

After Type 3, the first media sequence number is:

$$SEQB_1 = SEQZ + 1$$

The Stream Server reads the video in blocks of up to 1024 bytes and encrypts each block using the configured AES-GCM key.

Each Type 4 message includes an authenticated sequence number. The Proxy verifies the authentication tag and sequence number before forwarding the decrypted block to the browser.

Invalid, duplicated or out-of-order blocks are rejected.

3.6. Type 5 – End of Stream

The Stream Server sends an authenticated Type 5 message containing the next sequence number and an encrypted END marker with a random value.

The Proxy accepts the end of the stream only after validating the authentication tag, expected sequence number and END marker.

3.7. Type 255 – Error Message

Type 255 messages report protocol errors, including invalid handshakes, incompatible configurations and authentication failures.

Error alerts are encrypted and authenticated with AES-256-GCM, including sender identification as authenticated data.

A per-session replay guard rejects previously accepted alert nonces. Invalid or forged alerts are rejected.

4. Threat Model and Security Analysis

4.1. Attacker Capabilities

The network is considered untrusted. An attacker may eavesdrop, modify, inject, replay, reorder or drop messages exchanged between the Secure Proxy and Secure Stream Server.

The attacker controls the communication channel but not the legitimate endpoint systems. Endpoint compromise and direct attacks against correctly implemented cryptographic primitives are outside the assumed threat model.

4.2. Threat-Mechanism Mapping

The following table summarizes the security mechanisms and their limitations.

Threat	Mechanism	Limitation
Man-in-the-middle	Ed25519-authenticated X25519 exchange	Requires trusted public keys
Eavesdropping	AES-GCM encryption	Traffic metadata remains visible
Tampering / Forgery	AES-GCM authentication tags	Cannot prevent denial of service
Replay / Reordering	Authenticated sequence numbers and strict sequence checking	Invalid blocks interrupt streaming
Dropped messages	Sequence checking, authenticated Type 5 and connection-error detection	No automatic recovery of lost data
Invalid configuration	Handshake validation and authenticated Type 255 alerts	Only compatible configurations accepted
Session-key compromise	Ephemeral X25519 and independent session keys	Compromised session data is exposed
Endpoint compromise	Trusted-endpoint assumption	Outside the threat model
Primitive attacks	Secure primitive assumption	Outside the threat model

4.3. Session-Key Compromise and Forward Secrecy

Each connection uses fresh ephemeral X25519 keys and independently derived session keys.

Compromise of one session key exposes the data protected in that session but does not directly compromise independent sessions.

Ephemeral X25519 combined with long-term Ed25519 authentication supports forward secrecy against later compromise of identity keys, assuming ephemeral private keys have been erased and were not previously compromised.

5. Validation and Experimental Results

The system was validated through compilation, automated security tests and complete encrypted media transmission.

Tests covered authenticated key exchange, invalid signatures, modified ciphertext, message forgery, replayed and reordered messages, invalid sequence numbers and authenticated error alerts.

The tests confirmed rejection of invalid messages and incompatible cryptographic configurations.

AES-128-GCM, AES-192-GCM and AES-256-GCM were each tested through complete streaming sessions, successfully recovering the original 25,941,346-byte video without recompilation.

The decrypted video was also successfully played in a standard web browser. A complete transfer was verified by comparing SHA-256 hashes of the original and received files.

5.1. Streaming Performance

The following measurements were collected by the Secure Proxy during the complete playback of `dante.mp4` in a web browser, using localhost with zero artificial delay.

Metric	Measured Value
Encrypted blocks received	25,334
Decrypted blocks forwarded	25,334
Encrypted payload volume	26,853,370 bytes
Decrypted video volume	25,941,346 bytes
Original media size	25,941,346 bytes
Streaming duration	73.522 s
Observed throughput	352.84 kB/s (0.353 MB/s)

Throughput was calculated by dividing the decrypted video bytes forwarded by the elapsed processing time, using decimal kB (1 kB = 1000 bytes). The measurement covers the streaming phase after Type 3 until the authenticated Type 5 message.

The encrypted payloads contained 912,024 additional bytes, corresponding to 36 bytes of overhead per media block.

All 25,334 encrypted blocks were successfully decrypted and forwarded. The decrypted volume matched the original video size exactly, confirming complete media transmission.

These results reflect browser playback on localhost and may differ from measurements obtained through direct file downloads.

Throughput was calculated by dividing the decrypted video bytes forwarded by the elapsed processing time, using decimal kB (1 kB = 1000 bytes). The measurement covers the streaming phase after Type 3 until Type 5 validation.

The encrypted payloads contained 912,024 additional bytes, corresponding to 36 bytes of cryptographic and sequencing overhead per media block.

The decrypted volume matched the original file size exactly. The original and received SHA-256 hashes were also identical, confirming byte-for-byte equality.

These throughput results describe a local test environment and do not represent performance under real network conditions.

6. Conclusion

The Secure Streaming Solution provides authenticated key establishment, encrypted media transmission, message integrity, replay protection and sequence validation.

The secure channel successfully transmitted and recovered MPEG-4 media using AES-128-GCM, AES-192-GCM and AES-256-GCM, selected through XML configurations without recompilation.

Security tests and experimental measurements confirmed correct processing and rejection of invalid protocol messages. The recovered video was identical to the original.

References

[1] Computer Systems Security 2026/2027, *Mandatory Work Assignment #1: 3S – Secure Streaming Solution, Part I*.

[2] Go Standard Library, `crypto/ecdh`, `crypto/ed25519`, `crypto/aes`, `crypto/cipher`.

[3] H. Krawczyk and P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869.

[4] Elsevier Academic Journal template.